

36 — Chunking Strategy Comparison

What it is

When you store documents in a vector database for semantic search, you rarely embed a whole document as a single vector. Instead, you **chunk** (split) the document into smaller pieces and embed each piece separately. This is because embedding models have a limited context window, and because a single vector for a 5000-word document would blend too many topics into one representation — diluting the meaning.

But chunk size involves a fundamental trade-off:

- **Too small** (e.g. one sentence per chunk): each chunk is precise but loses surrounding context. A query about "industrial pump manufacturing capabilities" might not match a chunk that only says "We produce centrifugal models" because the chunk never mentions "pump" or "manufacturing."
- **Too large** (e.g. a whole 3000-word profile per chunk): the embedding captures everything, but irrelevant content drowns the signal. A query about "welding" gets a mediocre match because the chunk also talks about logistics, HR, and office supplies.

The **chunking strategy comparison** feature lets you empirically test this trade-off. It evaluates five strategies side by side:

Strategy	How it splits
Fixed 200 words	Sentence-boundary splits every ~200 words
Fixed 500 words	Sentence-boundary splits every ~500 words (the current production setting, R7)
Fixed 800 words	Sentence-boundary splits every ~800 words
Paragraph-based	Splits on double-newline boundaries (natural paragraph breaks)
Semantic (topic-shift)	Embeds consecutive sentences and splits when cosine similarity drops below 0.65

For each strategy, the system re-chunks a sample of documents, embeds every chunk, runs test queries against those embeddings, and measures retrieval quality (top similarity score, average top-5 score, document diversity in results).

Real-life analogy

Imagine you are preparing a book for a card-catalogue search system. You need to cut the book into index cards, each containing a portion of text.

- **Cut too fine** (one sentence per card): when someone searches "economic impact of railways," they find a card that says "Trains carried 40% of freight." Technically relevant, but the surrounding context — which decade, which country, what the comparison was — is on other cards. The searcher gets fragments, not answers.
- **Cut too coarse** (whole chapter per card): the card about "Transportation History" matches, but it also contains pages about canals, roads, and aviation. The railway content is buried in noise, and the match score is weaker than it should be.
- **The sweet spot** depends on your use case. If people search for specific technical specs, smaller chunks help. If they search for broad company descriptions, medium chunks preserve enough narrative flow.

The chunking comparison tool is like trying all three cutting methods on the same book and then running the same set of test searches to see which cutting method finds the right pages most reliably.

Why we used it here

Company profiles in this dataset range from ~50 words to 5000+ words. The ETL pipeline currently uses 500-word fixed-size chunking (requirement R7), chosen as a reasonable default. But is 500 words actually optimal for this dataset and these query patterns?

This comparison tool answers that question empirically rather than by guesswork. It:

1. Pulls real documents from the database (longest ones first, since short docs produce only one chunk regardless of strategy).
2. Re-chunks them using each strategy.
3. Embeds every chunk using the same embedding provider the production pipeline uses.
4. Runs a set of test queries (e.g. "industrial pump manufacturer," "welding services") and measures how well each strategy retrieves relevant content.

The metrics collected per strategy are: - **Total chunks produced** — more chunks means more embeddings (cost) but finer granularity. - **Average/min/max words per chunk** — shows the distribution shape. - **Average top similarity score** — how strong is the best match for each query? - **Document diversity** — how many unique documents appear in the top-10 results? (Higher diversity means the strategy is

not just returning multiple chunks from the same document.) -

Processing time — wall-clock cost of chunking + embedding.

This gives the team data to decide whether to keep 500-word chunks, switch to a different size, or even adopt paragraph-based splitting for this corpus.

Code walkthrough

Backend service: `search_api/services/chunking_comparison.py`

The STRATEGIES dict (lines 76-82) maps strategy keys to labels and splitting functions:

```
STRATEGIES = {
    "fixed 500": {"label": "Fixed 500 words", "fn": lambda t: chunk te
    "fixed 200": {"label": "Fixed 200 words", "fn": lambda t: chunk te
    "fixed 800": {"label": "Fixed 800 words", "fn": lambda t: chunk te
    "paragraph": {"label": "Paragraph-based", "fn": split paragraphs}
    "semantic": {"label": "Semantic (similarity)", "fn": _semantic_ch
}
```

Each entry's `fn` takes a string and returns a list of chunk strings. The fixed-size strategies delegate to the same `chunk_text()` function the production ETL uses (imported from `bilingual_etl/transform/chunker`), just with different word-count targets. This ensures the comparison is apples-to-apples with production behavior.

`_split_paragraphs(text)` (lines 31-33) is the simplest strategy — split on double newlines (`\n\s*\n`), strip whitespace, drop empties:

```
def split_paragraphs(text: str) -> list[str]:
    parts = PARAGRAPH_SPLIT.split(text.strip())
    return [p.strip() for p in parts if p.strip()]
```

This produces chunks of wildly varying sizes (a one-sentence paragraph vs. a 1000-word paragraph), which is part of the evaluation — does variable size help or hurt?

`_semantic_chunk(text, max_words=500)` (lines 45-73) is the most sophisticated strategy. It:

1. Splits text into sentences using regex (`(?<=[.!?])\s+`).
2. Embeds the first 50 sentences using the active embedding provider.
3. Walks through sentence pairs, computing cosine similarity between consecutive sentence embeddings.

4. Starts a new chunk when either: (a) the current chunk would exceed `max_words` , or (b) the similarity between consecutive sentences drops below 0.65 (indicating a topic shift).

```
for i in range(1, len(embeddings)):
    sim = cosine_similarity(embeddings[i - 1], embeddings[i])
    word_count = len(sentences[i].split())

    if current_words + word_count > max_words or sim < 0.65:
        chunks.append(" ".join(current))
        current = [sentences[i]]
        current_words = word_count
```

The 0.65 threshold is a tunable parameter — lower means fewer splits (bigger chunks), higher means more splits (smaller chunks). The `max_words` cap prevents a single topic from producing an unboundedly large chunk.

`_get_sample_documents(table, limit)` (lines 85-101) pulls sample documents from the database, ordered by length descending. This prioritizes long documents where chunking strategy actually matters (short documents produce one chunk regardless):

```
cur.execute(
    f"SELECT {id_col}, language, {text_col} FROM {table} "
    f"WHERE language = 'en' ORDER BY LENGTH({text_col}) DESC LIMIT %s"
    (limit,),
)
```

`compare_chunking_strategies()` (lines 118-215) is the main orchestrator. For each strategy:

1. **Chunk**: Run the strategy's splitting function on every sample document, collecting chunk stats (total count, min/max/avg word count).
2. **Embed**: Batch-embed all chunks (batches of 10) using the active embedding provider.
3. **Evaluate**: For each test query, embed the query, compute cosine similarity against all chunk embeddings, sort descending, and extract metrics from the top results.
4. **Aggregate**: Compute per-strategy averages (avg top score, avg document diversity).

The test queries are hardcoded in `TEST_QUERIES` (lines 104-109) — four English industry queries that represent typical search patterns.

The function logs an `EVIDENCE_CHUNKING_COMPARISON` marker for UAT traceability, then returns a structured dict with all results.

API endpoint: `search_api/routers/chunking.py`

A single GET endpoint with two query parameters:

```
@router.get("/admin/chunking-comparison")
def get_chunking_comparison(
    sample_size: int = Query(10, ge=1, le=50),
    table: str = Query("category_vectors"),
):
```

- `sample_size` : how many documents to sample (1-50, default 10).
- `table` : which vector table to pull from (`category_vectors` or `company_vectors`).

The endpoint excludes the `semantic` strategy by default (it requires many extra embedding calls), passing only the four faster strategies.

Frontend:

`admin-ui/src/pages/ChunkingComparison.jsx`

The component has three visual states:

1. **Empty state**: A scissors icon and instructions to click "Run Comparison."
2. **Loading state**: A progress bar and message warning it may take 1-2 minutes.
3. **Results state**: Three sections —

Stat cards (lines 76-88): Three summary numbers at the top — documents sampled, test queries run, strategies compared. These use the `.stat-grid` / `.stat-card` CSS classes from the shared admin UI styles.

Strategy summary table (lines 90-127): One row per strategy showing total chunks, avg words/chunk, min/max word range, avg top score (rendered as a gradient progress bar via `scoreBar()`), document diversity, and processing time. The `scoreBar()` helper (lines 25-43) renders a horizontal bar where width is proportional to the score:

```
function scoreBar(value, max = 1) {
  const pct = Math.round((value / max) * 100);
  // renders a gradient bar + numeric label
}
```

Per-query expandable details (lines 129-161): Each strategy gets a collapsible `<details>` section showing per-query breakdown — best

score, avg top-5, unique docs in top-5 and top-10. This lets you see which queries benefit most from a given strategy.

How to explain this in an interview / to a teammate

"When we embed documents for vector search, we have to split them into chunks first — but chunk size significantly affects retrieval quality. Too small and you lose context; too large and the embedding gets diluted with irrelevant content. Our project includes a comparison tool that re-chunks the same set of real documents using four different strategies — fixed 200, 500, and 800 words, plus paragraph-based splitting — embeds each set, runs the same test queries against all of them, and measures which strategy produces the strongest and most diverse matches. This gives us empirical data instead of guesswork when choosing a chunk size, and it runs as an admin endpoint so we can re-evaluate whenever the dataset changes."